

ABRA — Project Summary

Version 3.30.0 • 2026-07-31 • Will Hooper

A one-page map of the whole project and every component. For depth: the [white paper](#) (math + sources), the [deck](#) (plain-English), the [technical docs](#) (how to run it), and the living [model ledger](#).

What ABRA is

ABRA is a decision-support model family for **Pokémon Champions VGC, Reg M-B, best-of-one closed-sheet ladder**. It stores every public ladder replay and builds small, CPU-trainable models on that growing store. It runs in a browser with no build step.

The finding that shapes everything

You cannot reliably predict who wins from the two team sheets — even a player-rating model ties a coin. So ABRA does not sell outcome prediction. It supports *decisions* and grades every model with a proper score, a confidence interval, and an honest baseline. Wins are reported as wins; two honest negatives are reported as negatives.

The finding that shapes what gets built next (2026-07-30)

Four experiments added knowledge to the model. All four measured a null. Two experiments changed what the model is optimising for. Both were large wins.

change	kind	result
take the best move instead of sampling it	objective	+12 points, 79.7% of decisive pairs
self-play policy improvement over the clone	objective	55.9%
four separate feature additions	knowledge	four nulls

RECONCILED 2026-07-31. *That 55.9% was measured on the **53-feature vector with switching OFF**. Repeating the experiment on the **56-feature vector with switching ON** gives **48.1%** [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.*

The nulls survived the obvious check: an overdispersion test across teams reads ~1.00, against 1.169 for a known real effect, so they are genuine rather than a real effect hidden by team variety.

The constraint is the objective, not the knowledge. This is why the next item is retraining a model that already exists (DODUO, the pair-scoring layer, which lost at 42.0% fitted to *resemble humans* and has never been fitted to *win*), rather than adding more features to MAG.

A second, blunter lesson from the same day. Every integrity bug found had one shape: a fact reached one consumer and not the next. Priority blocking was in the artifact but not the simulator, so Sucker Punch beat a Farigiraf in every game ever simulated. A switch-in's own ability never reached the code that chooses the switch — measured over 40,001 matchups, declaring Intimidate, Drizzle or Drought changed nothing at all. And every mega forme carried no ability, no moves and no item, so **26% of the format scored as threatening nothing**. None of these were modelling disagreements. They were plumbing.

The components at a glance

Model	What it is	Status	Headline result
MEDICHAM	Hand-written doubles damage engine	⚠️ Being replaced	Within 5% of the Smogon calculator on 31 scenarios, but disagrees with the OFFICIAL Champions engine by 31.1 points of win probability. ADR-001: becomes a lookup over precomputed tables
GURU	Meta matchup matrix from real outcomes	⚠️ No decisive cells	Now quality-filtered (2026-07-25). On 1,124 clean games: 11 archetypes, 0 statistically-decisive matchups (n≥30, CI excluding 50%), and its predictive test scores 0.735 vs a coin 0.693 — worse than a coin. Descriptive structure only
XATU	Opponent set + next-move belief	✅ Built	Top-1 36% / top-3 72% on held-out human moves (beats its baselines)
PORY	Mid-game win-probability value net	⚠️ Contribution unclear	Log-loss 0.567 vs coin 0.693 — but its features ARE the material state, and it loses to a two-feature baseline (alive_diff+hp_diff 0.5822 vs PORY 0.5840, same estimator). Report the gain over MATERIAL, not over a coin. See engine/pory_baseline.py
CHOMP	Bring-4 / lead-2 team-preview engine	✅ Ships (standalone)	Exact-damage picker; CHOMP-EV proof: brings tie a coin (honest null)
SLOWKING	Team-preview Nash (mixed strategy)	✅ Built	Equilibrium << exploitable than uniform; playstyle cycle is suggestive on small samples
KADABRA	Replay coach	✅ Works offline	Per-turn "you're at X%" from PORY
DITTO	Team optimiser	⚠️ Pivoting	Objective de-biased to validated damage (was optimising a backwards signal)
ALAKAZAM	In-battle decision engine (capstone)	➡️ In development	Belief + search + learned value; built last on the inputs above
MEW	Self-play data engine	✅ Built	Runs the OFFICIAL Champions engine against itself on real observed teams. 1,000 games, 13/13 validation checks, mirror 51.0% CI [45.4, 56.6]
MAGNEMITE (MAG)	The in-battle policy that reads the board	Built, and improving by self-play (3.28.0)	Conditional logit over 53 features , fitted to 146,910 real human clicks from 6,091 clean open-sheet games. Held out by game: top-1 33.6% against the behaviour clone's 27.1%. It now DOES decide switches and DOES run a real damage calculation — both were listed here as missing and both became false. Still one ply, still no model of the opponent's move
DUSK	Endgame exact solver	➡️ Roadmap	Solves small boards (≤2v2, 1v1) perfectly — sharpens ALAKAZAM's endgame and gives clean training targets for PORY
HYPNO	Opponent read / exploitability dial	➡️ Roadmap	Estimates opponent strength + predictability; tells ALAKAZAM when to play safe (vs strong) or exploit (vs weak/predictable)
ROLES	Multi-label team composition (26 roles)	✅ Built	Role-pair matrix pools data to median cell n=20 across 1,051 cells (vs old single-label n=11–18) — the 7,971 once published was retracted in 2.7.0; preview roles tie a coin (honest null)
WAR	Wins Above Replacement (species RAPM)	⚠️ Null	Withdrawn 2026-07-25. Beat a coin only on the unfiltered store (0.6860). On clean games: 0.7048 vs coin 0.6931, accuracy 0.502 — the signal was four bots playing one team 1,446 times
NMF	Emergent roles / archetypes	⚠️ Rank not defensible	Rank 6 ships, but the project's own criterion (engine/nmf_rank.py , bootstrap factor stability, cf. Brunet et al. 2004) selects rank 4 — and

Model	What it is	Status	Headline result
			rank 6 scores -0.107 excess over null , i.e. its factors are <i>less</i> reproducible across resamples than factors fitted to shuffled data. The old justification here was reconstruction error 0.53, which that same script states cannot select a rank (it falls monotonically by construction). A team is a <i>blend</i> , learned not hand-labelled — but the number of blends is not currently defended

A phrasing the filter itself mandates. `require_full_bring` conditions on game length: measured 2026-07-31, the games it keeps are **1.71x longer** on average (7.4 vs 4.3 mean turns; 19,589 kept vs 8,713 dropped). Every bring statistic in this project is therefore "*the bring, among games long enough to show it*", which is not the same as "the bring". `data/quality-filter.json` states this at the point of filtering and requires it to be said downstream; this is that.

How it fits together

The **store** (every real game) feeds **GURU** (meta), **XATU** (belief), **PORY** (value), and **MEDICHAM** (damage). **SLOWKING** solves the preview; **CHOMP** picks the bring; **KADABRA** coaches a replay with PORY. **ALAKAZAM** is the capstone that assembles belief + search + value into the win-%-optimal move, built last. Every change updates the code, this summary, the white paper, the deck, the technical docs, and the CHANGELOG in the same pass.

Repositories and site

Piece	Repo	Live
ABRA (models + site)	<code>github.com/willhoop/abra</code>	<code>willhoop.github.io/abra/app/</code>
CHOMP (bring engine)	<code>github.com/willhoop/chomp</code>	Showdown userscript
Portfolio	<code>github.com/willhoop/willhoop.github.io</code>	<code>willhoop.github.io</code>

The data, as of 2026-07-25

Collected (closed-sheet Bo1 ladder)	see <code>data/live.js</code> — generated on every refresh, growing hourly (hardcoded sizes retracted, S13)
Usable after the quality filter	1,124 (12.8%)
Self-play (MEW, official engine)	1,000 — separate file, never pooled
Open-team-sheet archive	4,167 (MIT, 2026-06-17..20) — separate file, different information regime
Smogon official priors	283 species, whole-ladder aggregate

Two metagames, not one

`meta-usage.json` publishes both, because they answer different questions:

competitive	<code>garchomp, incineroar, kingambit, sinistcha, whimsicott, basculelion</code>
ladder	<code>garchomp, whimsicott, kingambit, basculelion, charizard, incineroar</code>

Competitive is what humans choose when trying — right for tournament prep and for any claim about the game. **Ladder** is what you actually face: three in four STORED games involve a bot. That is a property of what gets uploaded rather than of the ladder: bot-team species are over-represented in the scrape by a mean

of +8.3 points against Smogon whole-ladder statistics, while other top species run -2.9. The true share of bot opponents is lower than the store implies, but it is not small. Charizard sits at 25.7% on the ladder view and outside the competitive top six because it is on the bot team. Consumers must say which they used.

Honest ceilings

Predicting the match winner from sheets is a coin flip in this format — and the previously published 55.0% skill ceiling was itself measured with bots included. Removing them gives 52.4%, an interval that contains a coin flip. Every preview-level model now sits at that ceiling: JOLTEON, roles, CHOMP-EV, and as of 3.2.0 **WAR, whose result is withdrawn.**

Most results here are also **underpowered**: 1,124 clean games can only detect an edge of ~4.2 accuracy points, and a 2-point effect needs ~4,900. `engine/eval_harness.py` now refuses to report a null without stating what it could have seen.

The one load-bearing win is the **validated damage engine** — 31/31 within 2% against two independent oracles. PORY was the other, until 2026-07-25 showed it loses to a two-feature material baseline. The project's two genuine contributions are the ones it treats as plumbing: **behavioural bot detection**, and the **measurement discipline** that dissolved WAR, the 55% ceiling and GURU's matchup matrix in a single day.

Correction — the scrape over-samples bots

Measured 2026-07-25 against Smogon's whole-ladder statistics for the same format and month (1,163,315 battles vs the count generated into `data/live.js`):

	mean difference, uploaded vs whole ladder
The five bot-team species	+8.3 points
Every other top species	-2.9 points

75% of *stored* games involve a detected bot. That is a fact about **what gets uploaded**, not about the ladder — bots save replays far more readily than humans do. Any statement of the form "three in four of your opponents are bots" is therefore an overestimate and should not be made from this store.

This also bounds the earlier upload-bias result. Comparing our open-team-sheet Bo3 games against the whole Bo3 ladder gave a mean absolute difference of only 1.84 points — but that corpus contains almost no bots (29 named-bot sides in 4,167 games). So **human** upload bias is small; **bot** upload bias is large, and the closed-sheet Bo1 store carries the latter.